BINF-F402

MapQ project

Nicolas Roux
2024/2025

MapQ project

Table des matières

Introduction 2
Global structure of the tool 2
How to use 2
Summary 3
FastQ analysis 4
Mapping analysis 4
Genome reference 4
Mapping 5
Statistics 5

Introduction

The aim of the project is to create an R tool to analyse one or more FastQ files. The tool should provide several pieces of information about the quality of the raw reads and the quality of their alignment to a reference genome. It can perform the analysis for a single file or a comparative analysis for two or more files by simply providing the FastQ file(s) and the relevant genome.

Global structure of the tool

The tool is an R Markdown file. The users will need to install R Markdown in R studio to use it if they do not have it. The file begins with a chunk where the users can change the title of the report, their names, the date and the output options (it is not recommended to change them). Below this section, there are several chunks of R code to process the analysis. With the parameter echo = FALSE in the first chunk no line of code will appear in the HTML report, only the outputs. The outputs in the report will be:

- For the FastQ analysis: one table for all the files, one plot and one histogram side by side for each file.
- For the Mapping analysis: the same structure of the FastQ part - one table and two graphics.
- The running time.

There are seven chunks of code in the file:

- {r setup}: load the packages and downloading them if needed
- {r files}: load all the files needed to begin the analysis
- {r FastQ}: process the analysis of FastQ files.
- {r ref_genome}: the reference genome is loading and indexed at this step
- {r mapping}: process the mapping of the FastQ files to the reference genome
- {r mapping statistics}: process the mapping and the coverage statistics.
- {r time}: compute the running time.

The text between the chunks appears in the report as text.

How to use

To use this R tool, the users need to provide one file named samples.csv, which gives the basics information of the files. This file should include one column called Type, specifying whether the experiment was performed on RNA or DNA. This is important for the tool when using the align function from the Rsubread package for the mapping.

The samples.csv file should also include either two columns Read1 and Read2. If the experiment is about paired end reads and provides two files, the names of the files should be in the column Read1 and Read2. If the experiment is about single end reads then the file's name should be in Read1, the Read2 column should be filled with an NA value. The names of the files in these columns must include their extensions.

The samples.csv file must include a last column named « Id » containing the name of the experience: the tool will create a new bam file during the mapping step with the name provided in this column. This file can have other columns it will not interfere with the tool. The picture below gives an example of a properly formatted samples.csv file.

```
Read1,Read2,Id,Type,Cond
ENCFF215XOC.fastq.gz,ENCFF392RPJ.fastq.gz,ENCSR308HUY_rep1,rna,zfh2_16_18
ENCFF040HHO.fastq.gz,NA,ENCSR661BEZ_rep1,dna,CTCF-GFP
```

Figure 1 : Example of a samples.csv

If the users did not download the FastQ files in the same directory as the samples.csv file, there are two options:

- they can download all the files manually in the same directory
- they can create a CSV file named urls.csv with one column called **Urls** containing the URL of each file: the tool will read urls.csv and download all the FastQ files using the URLs provided.

The names of the FastQ files must be the same as in the samples file. The picture below gives an example of a properly formatted urls.csv file.

```
"Urls"
"https://www.encodeproject.org/files/ENCFF215XOC/@@download/ENCFF215XOC.fastq.gz"
"https://www.encodeproject.org/files/ENCFF392RPJ/@@download/ENCFF392RPJ.fastq.gz"
"https://www.encodeproject.org/files/ENCFF237XCS/@@download/ENCFF237XCS.fastq.gz"
"https://www.encodeproject.org/files/ENCFF760UCQ/@@download/ENCFF760UCQ.fastq.gz"
```

Figure 2 : Example of a urls.csv file

Finally, the users must have in the same directory the FASTA file of the reference genome needed for the mapping step.

When these conditions are met the tool can be use by providing in R studio the working directory where the samples, urls and genome files are. In the chunks named {r files} the first line store in an object the directory which the users need to specify. After that the tool should produce the report simply by clicking the knit button.

```
```{r files}
#Put your working directory file in the file_directory object or directly
in the setwd function
file_directory <- 'C:/Users/nicol/OneDrive/Bureau/master_1/Genomics
F402/MapQTest'
setwd(file_directory)
```

Figure 3 : code where to put the working directory

Summary

The users will need a **samples.csv** file and the reference genome in a FASTA format zip or not. They can add a **urls.csv** file but is not mandatory. The working directory needs to be copy and paste in the file directory (figure 3). When these steps are complete the analysis can be started by clicking the knit button on top of the script (figure 4).
```

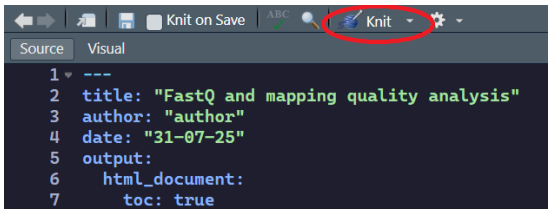

Figure 4 : screenshot of R studio. The knit button is on the red circle

It will provide an HTML report with two tables about the basics information of the FastQ files and the statistics about the mapping and the coverage processed by the tool, two graphics per FastQ files and two graphics per mapping.

FastQ analysis

For the analysis of the FastQ files the tool creates one table and two graphics per file within a for loop. To make the two graphics side a side it needs the graphics to be created with the ggplot package to use the grid.arrange function from the GridExtra package. This function put the two graphics on the same line. With Rmarkdown this was the only way to put these graphics that way, while the par and layout function wasn't working here.

The table, initialized before the loop, provides the basic statistics of the FastQ files like the number of raw reads, the type of reads from the experience (RNA or DNA), the mean size of the reads (this line provides the length of the reads if they all have the same length) and the mean of the Q-score in each files. To provide these information the tool reads the FastQ file with the readFastq function from the ShortRead package. The mean length and the mean quality were calculated through the sread function to extract the sequences of the reads and the quality function to get the Q-score of each nucleotide of each read. The tool will put all the Q-score with the quality function in a large matrix where each column is a position in the read. With this matrix a first graphic is made by calculating the mean Q-score by position the read. At first the tool should have done a boxplot of the Q-score by position, but it caused RStudio to become unresponsive, preventing the continuation of the analysis. A red line is added to the graph and corresponding to the probability to have exactly 0 error in a read considering the length of the reads. It provides an estimate of the minimum quality needed for reads of a given length to remain reliable.

This graph is followed by a histogram to show the distribution of the Q-scores of the reads in the file. The Q-score of the reads is calculated by doing using the mean function on the lines of the matrix containing all the Q-scores. It shows the global quality of the reads in the files and it indicates if the file should be filtered to remove low-quality reads.

Mapping analysis

Genome reference

To map the fastq files, the tool will first load and index the reference genome with the functions readDNAStringSet and buildindex from the Biostring and the Rsubread package. The names of the chromosomes from the genome will be loaded in a vector to be able to compare the mapping statistics across the chromosomes.

Mapping

Now that the genome is loaded, the mapping of all the files is done with the align function in an mapply function. It will generate a BAM file with the name provided in the Id column followed by « _mapped.bam ». In this function the type of reads is check first because the align function needs one or two parameters considering if it is single end or paired end reads. The number of threads is set to 4, but it can be raised manually to speed up the process if the user's computer allows it.

```{r mapping, include=FALSE}
mapply(FUN = function(file1, file2, nt_type, r_type, id) {
  if (r_type == 'single_end') {
    align(index = gsub('\\.(fa|fasta)(\\.gz)?$', '_index',genome_id),
          readfile1 = file1,
          type = ifelse(nt_type == 'rna', 'rna', 'dna'),
          output_file = paste(id, "_mapped.bam", sep=""),
          nthreads = 10)
  } else {
    align(index = gsub('\\.(fa|fasta)(\\.gz)?$', '_index',genome_id),
          readfile1 = file1,
          readfile2 = file2,
          type = ifelse(nt_type == 'rna', 'rna', 'dna'),
          output_file = paste(id, "_mapped.bam", sep=""),
          nthreads = 10)
  }
}, samples$Read1, samples$Read2,samples$Type, samples$Reads_type,samples$Id)
```

Figure 5 : R code from the MapQ tool showing how the files are mapped.

Statistics

After the mapping, the tool starts the analysis of the mapping and the coverage in a for loop. Every iteration performs the analysis of one mapped file provided earlier. Before that, a table with the future information is initialized and will be completed throughout the for loop. At the beginning of the loop the BAM file is sorted by the position of the reads with the sortBam function from the Rsamtools package. Then the new sorted file is indexed with the indexBam function. This process is needed to look at the coverage through the coverage function and to speed up some other manipulations.

To check the quality of the mapping there is the MapQ score which can found in the generated BAM file. It represents the probability of a read being misplaced. To analyse the quality of the mapping, the tool will look at the distribution of the Mapq scores of the reads in the files and generate a histogram with the frequency of each MapQ scores of the reads in the BAM file. The mean MapQ score in the file is also generated and added to the table. The proportion of the mapped reads can also give some information about the mapping quality: if the mapping is good, the expected proportion of mapped reads should be high. This information is added in the table with the number of reads in the file and the proportion of the mapped reads. The number of reads can be obtained when the file is loaded with the scanBam function from Rsamtools.

Next the tool will look at the coverage of the file. The coverage is the number of reads per position in the genome and can be obtained with the coverage function. The coverage throughout the genome is stored in an object which will be converted into a vector to access it more easily. It can now compute the average coverage of all the chromosomes and is done in a sapply loop with the mean function. To compute the average coverage of the genome, the sum of the coverage across the chromosomes is divided by the length of the genome. After that, a graphic of the coverage across the genome is provided to get an overview of coverage throughout the genome. To make this process easier and faster, the distribution of the local coverage is divided into 1000bp windows. This part of the code was taken from practical 4 of the BINF-F402 course.

```
compute_cov <- function(w, wsize) {
  w <- as.numeric(w)
  bins <- ceiling((1:length(w)) / wsize)
  return(as.vector(rowsum(w, bins)))
} #source Practical 04
cov_1kb_all <- lapply(chromosomes, function(chr) {
  w <- as(gcov[[chr]], "vector")
  compute_cov(w, 1000)
})
names(cov_1kb_all) <- chromosomes
```

Figure 6: R implementation of compute_cov. It computes the total coverage per 1000bp windows

A data frame of two columns with the position and the local coverage is made to get the plot with the ggplot package.

The for loop ends by putting the two graphics side by side with the grid.arrange function like in the FastQ part and by adding all the information to the table. The parameter fig.show='hold' in the chunk is there to put the table before the graphics (this parameter in the FastQ chunk has the same purpose).

The report ends by providing the running time of the tool with the Sys.time function. The running time is provided by computing the difference of the time at the end of the code and the time at the start of the code in the chunk where the packages are loaded. This information can be interesting for the users, especially if he needs to run this tool again. In this case he knows the time needed to analyse his files.